GENERATIVE AND AGENTIC AI · TOPIC 4

Customising LLMs

From Prompting to Fine-Tuning, Efficiently, on Modest Hardware

Laura Wynter, PhD
SMU School of Computing and Information Sciences
IS469 · AY2026–27

ROADMAP

What we'll cover

01 The 2026 customisation landscape, a ladder, not a default

02 Knowledge gap vs behaviour gap, when fine-tuning still wins

03 Training libraries & the HF stack

04 Supervised fine-tuning (SFT)

05 Beyond SFT: preference & reinforcement fine-tuning (DPO / GRPO / RFT)

06 Stable training, distributed training

07 PEFT & LoRA · mixed precision · quantisation

08 QLoRA, distillation & what's next

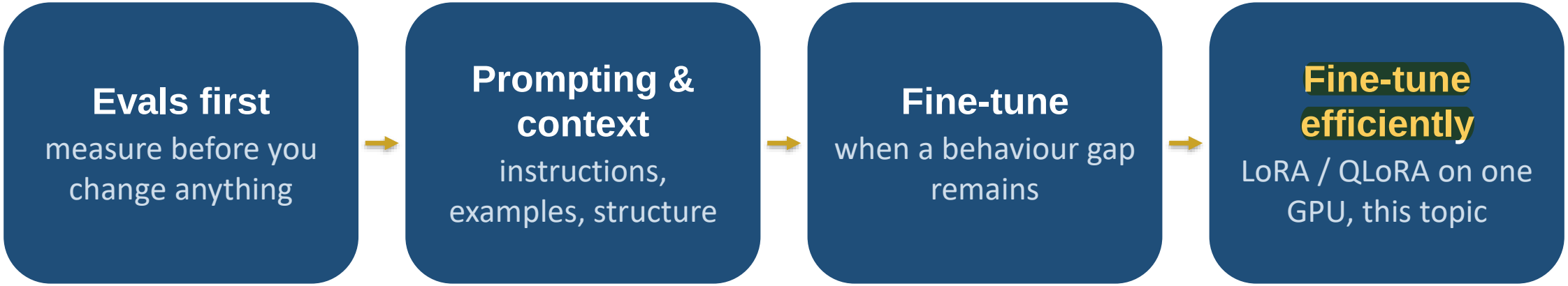
09 Hands-on lab (unsloth + TRL)

01

The 2026 Customisation Landscape

Fine-tuning is no longer step one, it's the last rung of a ladder.

The customisation ladder



Climb only as far as your evals force you to. Each rung costs more and is harder to reverse than the one below it.

Each rung costs more, and locks you in more



Schematic: prompting and context are cheap and reversible; a full fine-tune is the biggest commitment. Efficient fine-tuning is this topic's focus.

Knowledge gap vs. behaviour gap

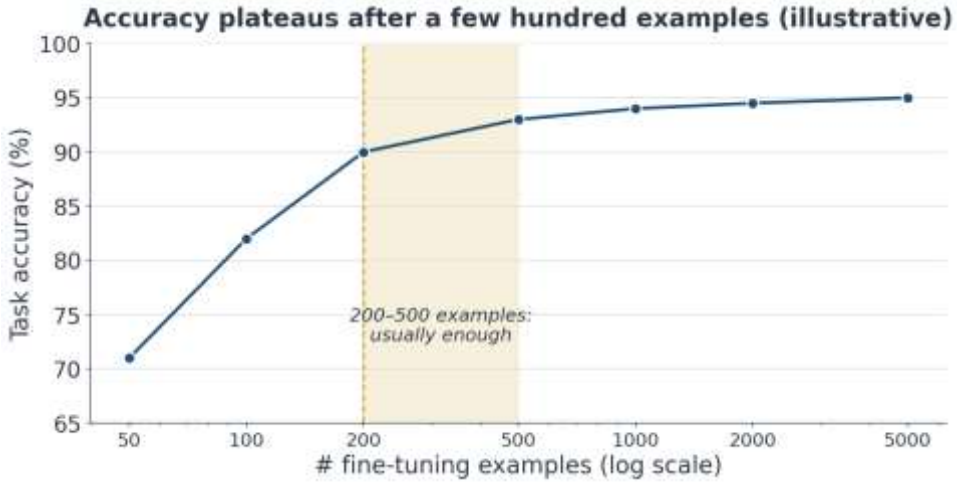
- First ask what is actually failing, the fix depends on the kind of gap:
  - Missing / stale / private facts → a KNOWLEDGE gap → fine-tuning is the wrong tool for it
  - Wrong format, tone, policy, or unreliable reasoning → a BEHAVIOUR gap → fine-tune
- Volatile knowledge (prices, regulations, docs) should never be fine-tuned in, it goes stale
- Fine-tuning teaches a model how to behave, far more than what to know
- Exception: a task with your OWN label set (classification, ranking) is neither, you train for it directly

Where fine-tuning still clearly wins

- Strict output structure, JSON / schema adherence, house style, persona
- Latency & cost, distil a task into a 1–8B model instead of calling a frontier API
- Verifiable skills, math, code, structured extraction, reliable tool-calling
- Domain behaviour encoders still almost always need it (classification, retrieval, ranking)
- Private data that cannot leave your infrastructure at all

How much data? It depends on the task

- No single number, it scales with what the model actually has to learn
- Style, tone or output format: a few hundred clean examples often suffice
- Classification is NOT a behaviour task: budget per CLASS, not per dataset
  - ~50–100 per class: 5 classes ≈ a few hundred, 100 classes ≈ thousands
- The curve on YOUR data says when to stop, not a rule of thumb



Illustrative, diminishing returns set in early.

02

The Efficiency Problem

Once you've decided to fine-tune, GPU memory, not compute, is the wall.

The efficiency problem

- Once you decide to fine-tune, the main bottleneck is GPU memory, not compute
- 2026 open-weight families ship in many sizes: Qwen3, Llama-4, Gemma-3, DeepSeek, Phi-4, MoE is now the default frontier open architecture
- Full fine-tuning of even a few-billion-param model overruns a single consumer GPU
- The rest of this topic: the techniques that make customisation fit on one GPU
- PEFT / LoRA, mixed precision, quantisation, and the newer preference / RL methods

Full fine-tuning vs. efficient customisation

Full fine-tuning (FFT)

- Updates every parameter of the model
- Optimiser states dominate memory (~16 bytes/param)
- Needs multi-GPU or datacentre hardware for large models
- One full copy of weights per fine-tuned task

Efficient customisation

- Trains a tiny fraction of parameters (PEFT / LoRA)
- Cuts precision: mixed precision + quantisation
- Fits a multi-billion-param model on one GPU
- Small, shareable adapters instead of full checkpoints

03

Training Libraries & the HF Stack

The tools you'll actually reach for, and what each one is good at.

Main libraries for LLM training

- Hugging Face, the default high-level training stack (we used it last time for BERT)
- PyTorch FSDP (Fully Sharded Data Parallel), distributed training across many GPUs
- DeepSpeed (Microsoft), distributed training + inference; still actively maintained and widely used
- In practice you compose these, HF on top, FSDP or DeepSpeed underneath for scale

The Hugging Face ecosystem

Library	What it does
Transformers	Load & train models, the core library
Accelerate	Thin layer for distributed / mixed-precision training
PEFT	Parameter-efficient fine-tuning (LoRA and friends)
TRL	Post-training: SFT + preference (DPO) + reinforcement (GRPO)
Hub	Hosts most open-weight models (audio & vision too)

TRL is now a full post-training library, one Trainer per method (SFT / DPO / GRPO), all on top of Trainer + Accelerate.

Transformers: Trainer & SFTTrainer

- pipeline(), quick out-of-the-box inference, but not for serious training
- Trainer, a wrapper over the PyTorch training loop; sets hyper-parameters cleanly
- SFTTrainer (TRL), a wrapper over Trainer specialised for supervised fine-tuning
- Handles loading model + dataset, tokenising, batching, the data collator, and training args
- Good balance of flexibility and ease-of-use, the default we build on in the lab

04

Supervised Fine-Tuning (SFT)

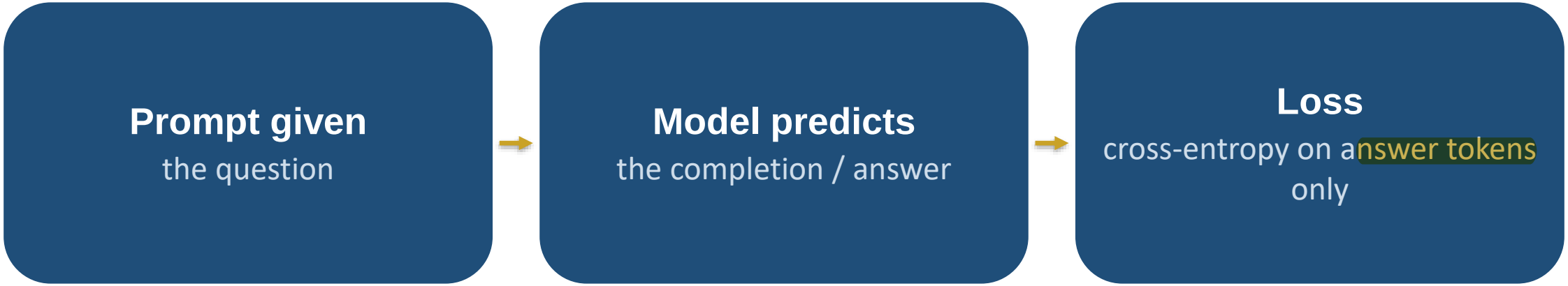
Teaching a model to produce answers, while only scoring the answer.

SFT and train-on-completions-only

Train on completions: We compute loss on o/p predicted not input

- The workhorse of fine-tuning: show the model labelled prompt → answer pairs
- The prompt (question) is given; the model must predict the completion (answer)
- Still cross-entropy loss, but we do NOT score the tokens in the prompt
- Only the predicted answer tokens count toward the loss
- In TRL this is the train_on_completions_only behaviour (mask the prompt tokens)
- API note (2026): SFT args like dataset_text_field / max_seq_length now live in SFTConfig

Train-on-completions-only



Prompt tokens are masked, only the predicted answer tokens count. *In TRL: train_on_completions_only.*

What SFT cannot do

- SFT imitates demonstrations, it copies the answers you show it
- It has no notion that one answer is better than another, only "this is the target"
- So it can't directly optimise for helpfulness, safety, or a preferred style
- For that we need preference and reinforcement methods, the next section

05

Preference & Reinforcement Fine-Tuning

The 2025–26 shift: optimise for better answers, not just imitate given ones.

Beyond imitation

- Post-training pipelines now go: pre-train → SFT → preference / RL alignment
- Preference tuning learns from comparisons: answer A is better than answer B
- Reinforcement fine-tuning (RFT) rewards good behaviour and penalises bad, over many tries
- This is how frontier models are aligned, and increasingly how teams specialise open models
- All of it composes with LoRA / QLoRA, you rarely update the full model

SFT → DPO → GRPO at a glance

Method	Learns from	Needs a reward model?
SFT	Labelled prompt → answer pairs	No
DPO	Preference pairs (chosen vs rejected)	No, closed-form update
RLHF (PPO)	Reward model trained on preferences	Yes
GRPO / RFT	A reward signal per sampled answer	Often no, rule-based reward

The trend is away from a separate reward model, toward direct (DPO) or rule-based / verifiable (GRPO) rewards.

Direct Preference Optimization (DPO)

- You collect pairs: for a prompt, a preferred (chosen) and a dis-preferred (rejected) answer
- DPO derives a closed-form loss straight from those pairs, no RL loop, no reward model
- Simpler and cheaper than RLHF; offline (no sampling during training)
- Used to post-train Llama 3 and many other open models
- In HF this is TRL's DPOTrainer, the default preference method

Reinforcement fine-tuning with verifiable rewards

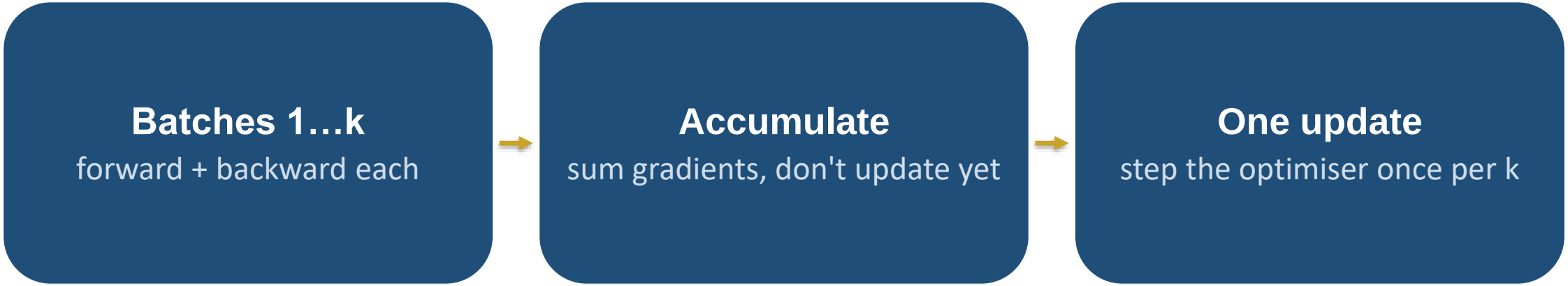
- RFT / RLVR = reinforcement fine-tuning where the reward is a checkable rule, not a learned model
- For math / code / structured extraction / tool-calls you can just check: is the answer correct?
- The model samples answers, correct ones are reinforced, it learns the behaviour, not one label
- GRPO (DeepSeekMath; popularised by DeepSeek-R1, early 2025) is the workhorse algorithm
- Payoff: a 7–13B model tuned this way can rival a 70B frontier model on its narrow task

06

Stable Training on Small GPUs

Bigger effective batches on small GPUs, and keeping gradients under control.

Gradient accumulation



Effective batch = per-iter batch × accumulation steps × num procs. A bigger effective batch means a less noisy gradient, so the learning rate must be scaled up to match it.

Using grad accum well

- Scale the learning rate **up** to match the larger effective batch: a bigger batch gives a less noisy gradient, so a larger step is safe and needed to cover the same ground in fewer updates
- Common rules: scale the LR linearly with the effective batch, or by its square root when using Adam / AdamW
- Watch convergence, reduce accumulation steps if training is too slow or erratic
- It's not free: gradients must be stored until the update, which also uses memory

Reducing exploding gradients

- Exploding gradient = a large spike in the gradient norm → unstable training, can miss the optimum
- L2 regularisation ("weight decay"), shrinks weights overall, reducing gradients
- Gradient clipping, cap the gradient norm at a threshold during back-prop, then update
- Both are cheap defaults; clipping in particular is standard for LLM fine-tuning

07

Distributed Training Paradigms

When one GPU isn't enough, the four ways to split the work.

§7 · IN BRIEF

Training across many GPUs

- When a model or batch is too big for one GPU, training is distributed, recognise these by name:
  - Data parallelism (DDP): replicate the model on each GPU, split the data
  - FSDP / DeepSpeed ZeRO: shard parameters, gradients and optimiser states across GPUs
  - Pipeline & tensor parallelism: split layers across GPUs, or split within a layer
- The internals are graduate-level; here it is enough to know that distributed training exists, and why

08

PEFT & LoRA

The observation that changed fine-tuning: you don't need to move every weight.

Parameter-Efficient Fine-Tuning

- PEFT rests on the observation that LLMs are heavily over-parameterised
- LoRA, "Low-Rank Adaptation of Large Language Models" (Microsoft Research, 2021)
- Later methods have not displaced LoRA, except QLoRA (2023)
- DoRA (2024) applies LoRA to the direction of each weight, a refinement with modest uptake
- Instead of full fine-tuning (FFT), update only a small number of parameters
- The payoff: fine-tune much larger models on a single GPU

How LoRA works

train B and A then multiply —> train a smaller number of parameters.

big alpha infront of BA if u think ur new training is important —> to put more emphasise—> alpha is bigger

- Full fine-tuning computes $(W_0 + \Delta W)x$, with ΔW the same shape as W_0
- LoRA factorises it: $\Delta W = BA$, rank $r \ll d$, and trains only A and B
- The layer computes $(W_0 + BA)x$; W_0 frozen, folding them back is "merging"
- Why two small matrices rather than one dense ΔW?
  - **Memory**: only the $d \times r$ and $r \times d$ shapes, and their optimiser states, are stored
  - **Hardware**: Tensor Cores are built for dense matrix multiplication, which BA is
  - **Expressiveness**: ΔW is dense, so every one of its $d \times d$ entries can move, yet only within a low-rank subspace, letting learning spread across the whole layer

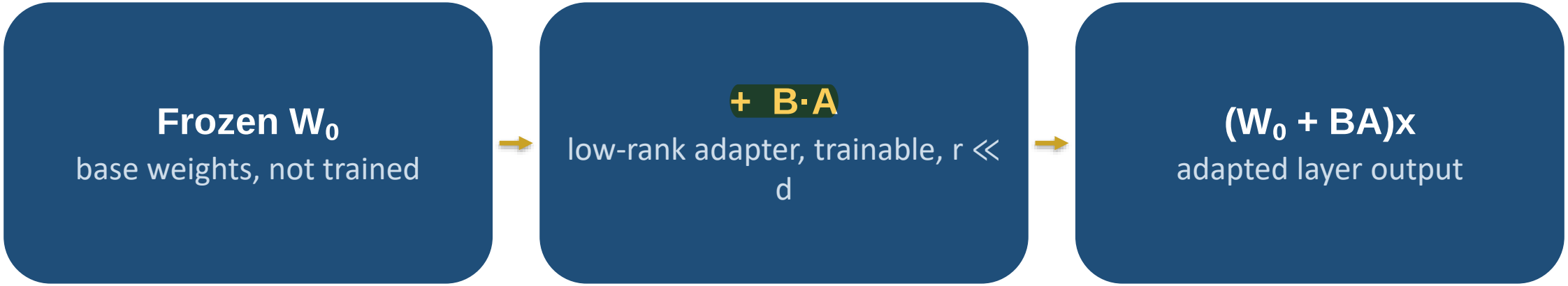
The low-rank adapter

By traning smaller amount of parameters, enough to learn vast information and we are not destroying the old knowledge

Reasons why this works:

1. neural networks are over parameterized, by training a small percentage of them still can get extremely good results

2. By not training all of the parameters, we are not destroying the old knowledge

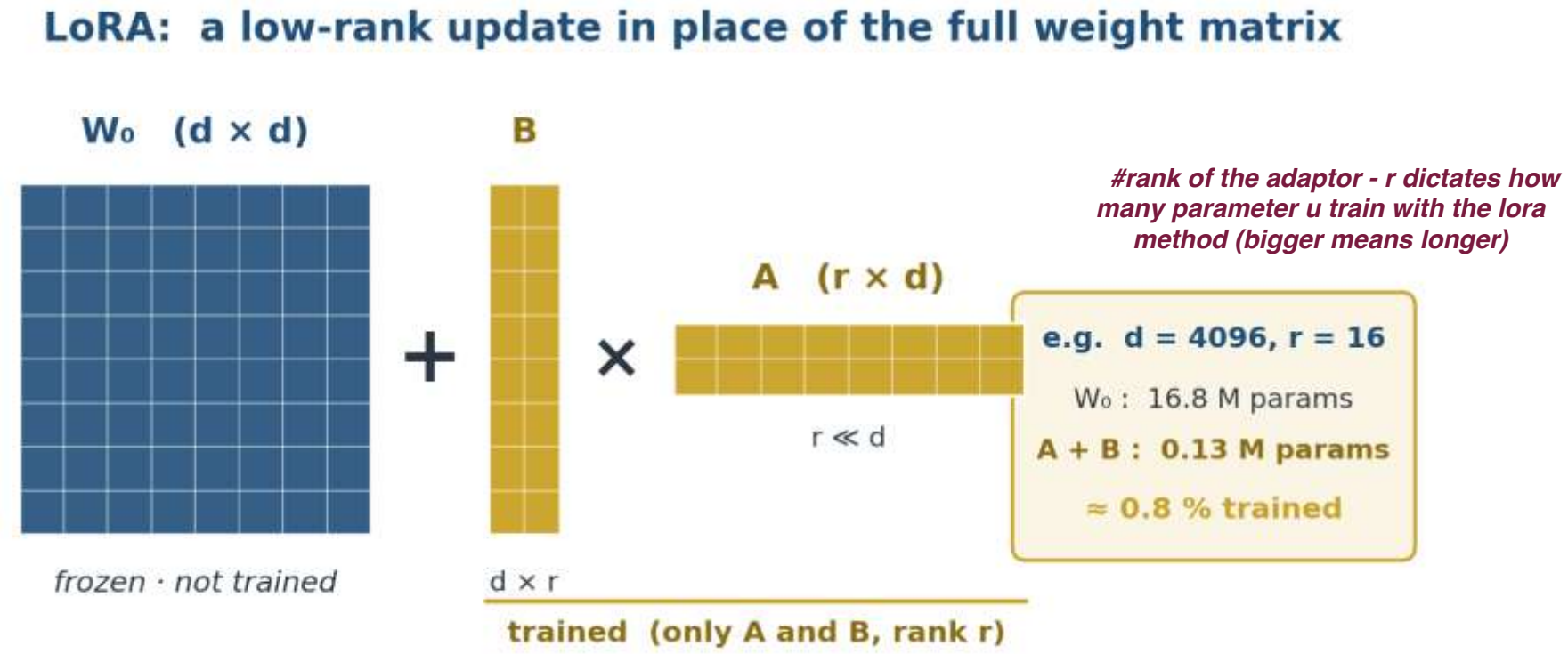


Only A and B are trained (rank r ≪ full dimension) → trainable params collapse to ~0.01%.

n.o step in epoch is n.o ex in dataset/ batch size

The low-rank update, as matrices

rather than training all of the weights, define two matrix which are much smaller then train those. Multiply them and add to original weights

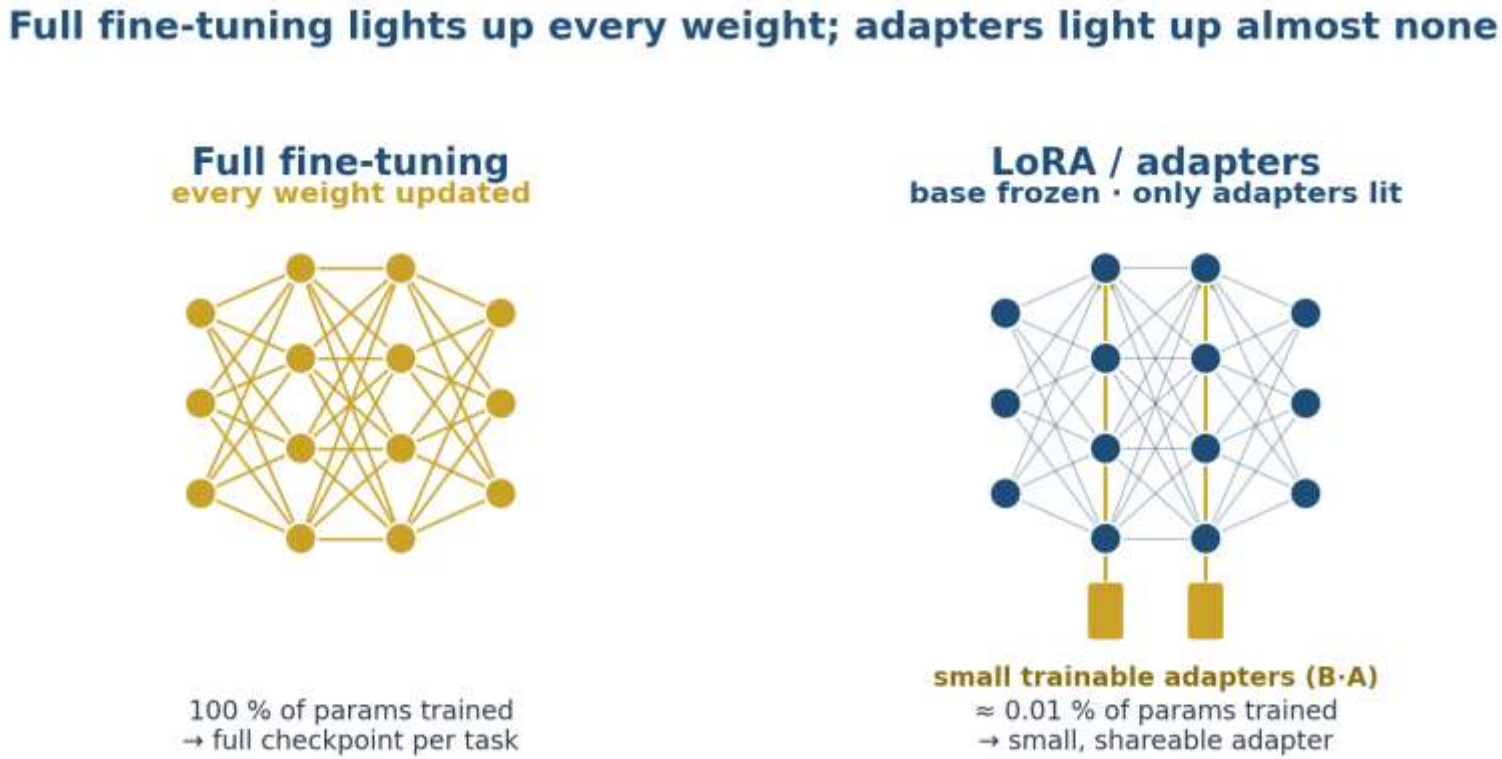


ΔW is never formed at full rank: the thin B (d×r) and A (r×d) are the only trained tensors, here ~0.8% of W_0's parameters. Illustrative.

LoRA adapters in practice

- The base model's weights are frozen, never modified during LoRA training
- Only the A, B matrices are trained → the result is a small set of "adapters"
- Typically update ~0.01% of the original parameters (you choose which layers)
- Adapters are stored separately from base weights, pluggable and shareable
- In HF / unsloth the model is patched in place → reload the base each run, or adapters stack

Full fine-tuning vs. adapters, what actually trains



Full fine-tuning updates every weight (a full checkpoint per task); LoRA freezes the base and trains only tiny adapters, on the order of 0.01% of parameters.

LoRA hyper-parameters

Hyper-parameter	Meaning	Typical
r	Rank of the update matrices, lower r = fewer trainable params	16
target_modules	Which layers get LoRA matrices ("all-linear" = every linear layer)	"all-linear"
lora_alpha	Scaling factor, update is (α / r)·(BA)x	α = r

These three are the knobs you'll actually tune in the lab.

09

Mixed Precision

Store and compute in fewer bits, without wrecking the training.

Mixed precision: the memory picture

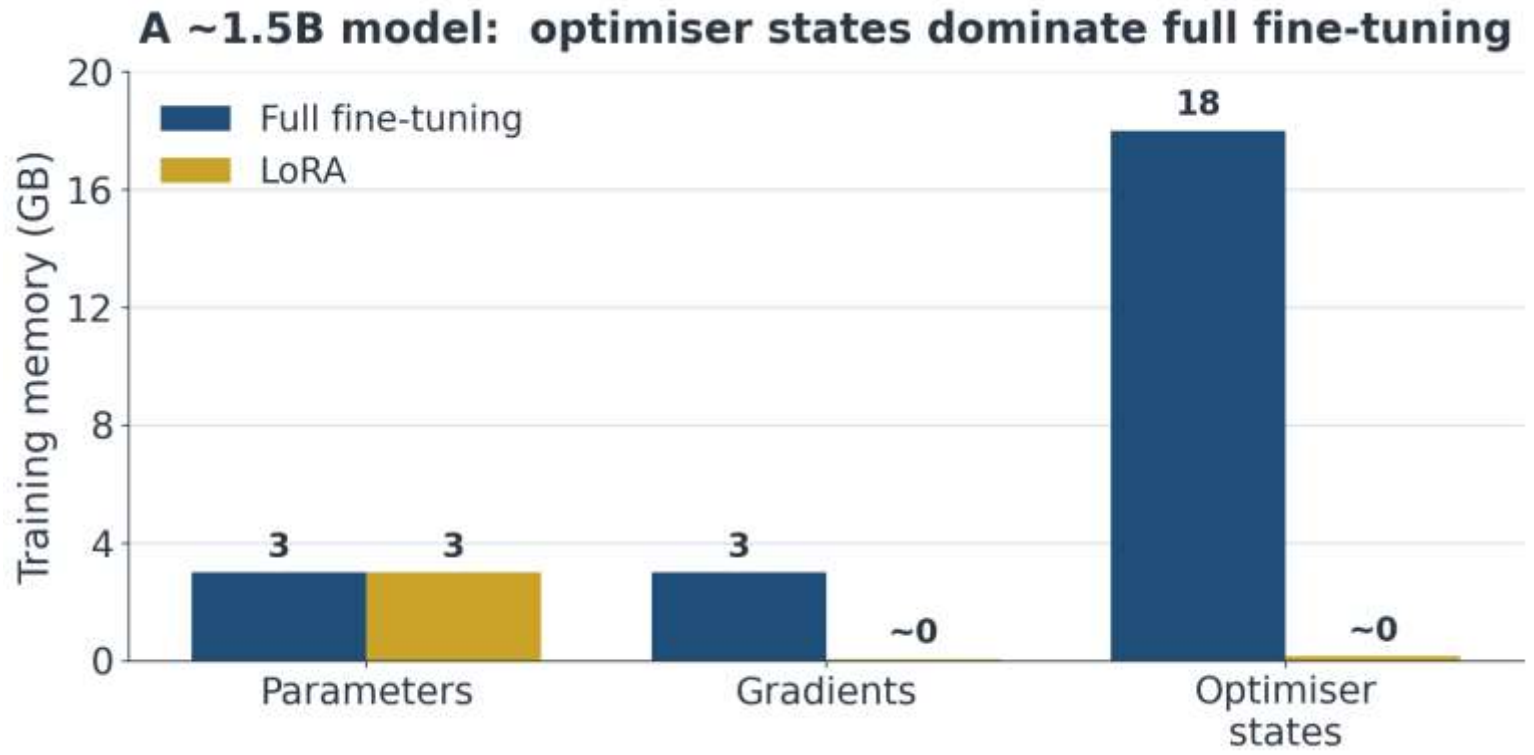
how we represent all of the numbers in the transformer

- Industrial GPUs (A100) have 80 or 40 GB; consumer GPUs often only 24 GB
- GPT-2 XL (the largest GPT-2, 1.5B params) needs ~6 GB for weights in FP32, ~3 GB in FP16
  - By contrast GPT-2 "small" is only 124M params, weights ≈ 0.5 GB in FP32
- But training needs far more: optimiser states, gradients, activations and buffers
- AdamW alone stores momentum + variance of the gradients per parameter
- Mixed precision = combine FP32 and FP16 to cut memory while keeping quality

Where the memory actually goes

- For N params in FP16 (2 bytes): params + gradients + optimiser states
- AdamW optimiser states = FP32 param copy (4) + momentum (4) + variance (4) = 12 bytes
- Total ≈ 16·N bytes just for model + states, e.g. a 1.5B model ≈ 24 GB before activations
- Activations (cached forward tensors) add more, scale with layers, seq length, batch, hidden, heads

Where training memory goes: FFT vs LoRA



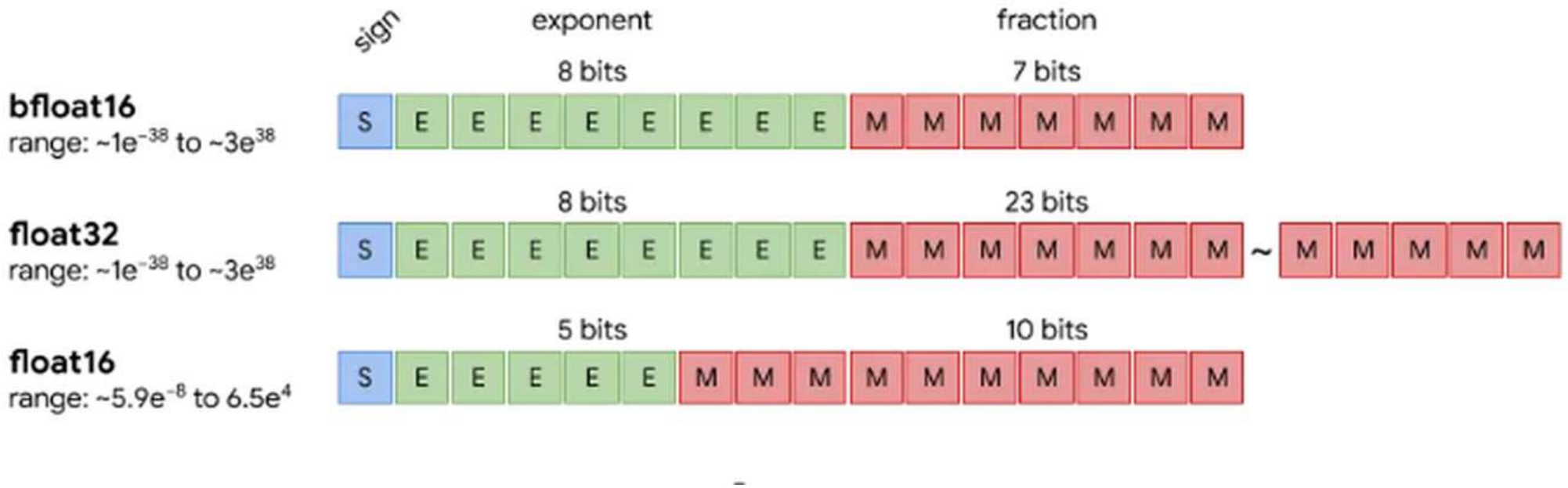
Optimiser states dominate full fine-tuning; LoRA freezes the base, so its gradients & optimiser states nearly vanish.

AMP, FP16 vs. BF16

how we represent all of the numbers in the transformer

- AMP = automatic mixed precision, match each op to its best datatype
- Matrix multiplies are fine in low precision; norms and exp usually need high precision
- Dynamic loss scaling shifts gradients into the representable range (avoids over/underflow)
- BF16 has FP32's dynamic range with less precision (more exponent, fewer mantissa bits)
- → fewer NaNs/Infs than FP16; BF16 is just the top 2 bytes of an FP32 number

What the bits actually hold: FP32, FP16, BF16



Sign, exponent, mantissa. BF16 keeps FP32's 8 exponent bits (same range, far fewer NaNs) and spends only 7 on the mantissa; FP16 has 5 exponent bits, so more precision but a much narrower range. Source: moocaholic, "FP64, FP32, FP16, BFloat16, TF32...".

10

Quantisation

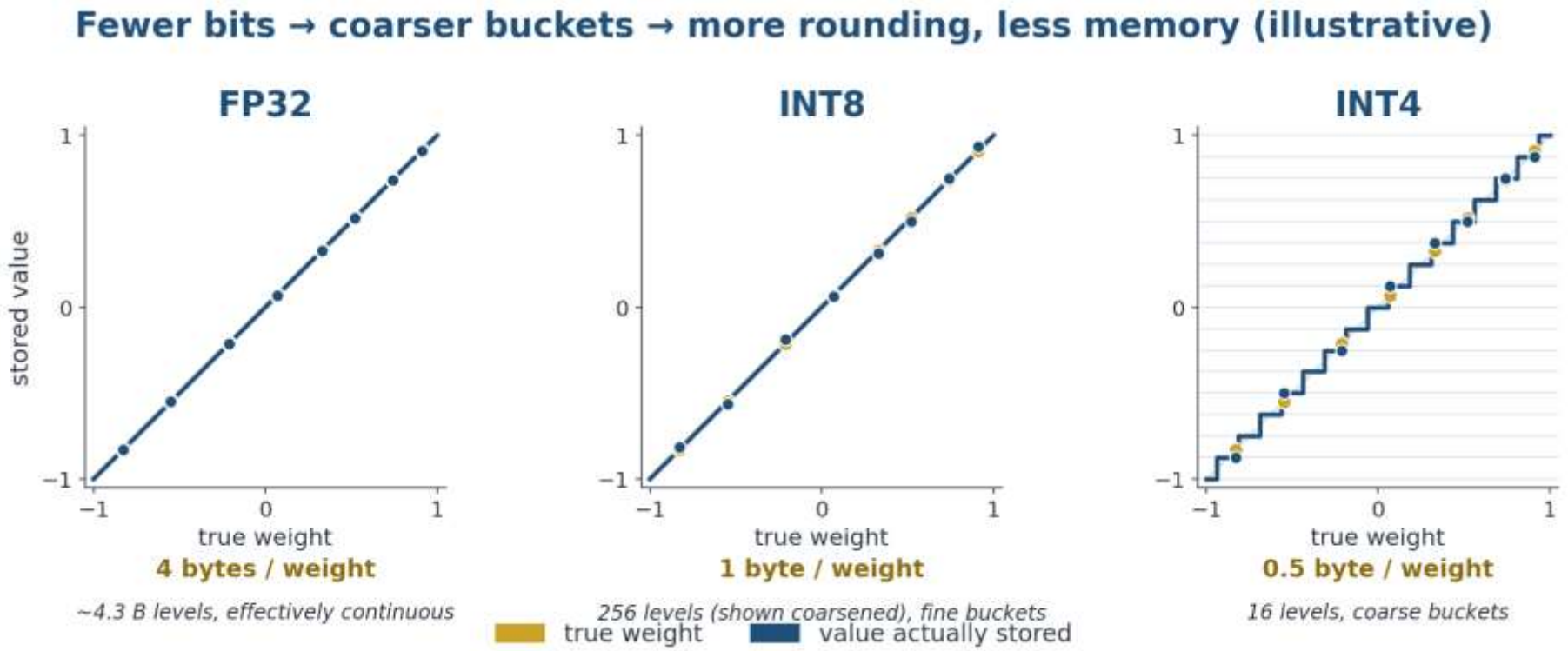
Fewer bits per number, the training-side view (Topic 5 owns inference).

What quantisation is

using fewer bytes to rep all of the number —> takes less space

- Quantisation stores numbers in fewer bits → less memory and lower compute cost
- Represent weights (and activations) in low-precision types instead of FP32 / FP16:
  - int8 (8-bit integer), int4 (4-bit integer), fp8 (8-bit float)
- We focus on quantisation for training (e.g. QLoRA); inference formats come in Topic 5
- 16-bit half precision is already a standard IEEE representation, sign · exponent · mantissa

Fewer bits = coarser buckets



The same real weights snapped onto FP32 / INT8 / INT4 levels: fewer bits means fewer buckets, so each value rounds further from its true position, the memory-vs-accuracy trade. Illustrative.

int8: symmetric vs. asymmetric

1bit can only rep 0 or 1
*3 bit—> 2**3=8 can represent 8 numbers*

scaling down representation such that it can be represented by just 8 bytes.

QUANTISATION: using fewer bytes to rep all of the number —> takes less space

- int8 dynamic range is [-127, 127], real values must be projected into that range
- Symmetric: scale by the largest absolute value α → a linear map centred at zero
- Asymmetric: use the actual min and max as the range (adds a zero-point offset)
- Symmetric quant: $X_Q = \text{round}(X \cdot 127 / \max|X|)$
- Symmetric dequant: $X_D = (\max|X| / 127) \cdot X_Q$

representing number using 8 bit

some information lost due to rounding

De-quantisation & preserving accuracy

- Some ops must run in higher precision → de-quantise int8 back to FP16 / BF16 / FP32
- Naive de-quant just re-adds zeros; storing scale + zero-point makes it far less lossy
- bitsandbytes (int8 / int4) stores extra tensors, "weight" and "SCB", to de-quantise
- Rounding loss still remains: quant rounds, and that rounding can't be recovered on de-quant

fp8: e4m3 and e5m2

- fp8 arrived with newer GPUs (NVIDIA H100), two variants: e4m3 and e5m2
- e5m2 is essentially a truncated FP16 (more exponent, less mantissa)
- Forward activations & weights need precision → use e4m3
- Gradients need dynamic range more than precision → use e5m2
- fp8 range fits any single tensor, not all at once → per-tensor scaling factors
- DeepSeek 3.1 introduced an fp8 ue8m0 scaling variant (for new Chinese GPUs)

Number formats at a glance

Format	Bits	Use	Note
FP32	32	Master weights / optimiser	Full-precision baseline
FP16	16	Mixed-precision compute	Narrow range → can over/underflow
BF16	16	Mixed-precision compute	FP32 range, fewer mantissa bits
int8	8	Quantised weights	Symmetric or asymmetric scaling
nf4	4	QLoRA frozen base	"Normal float", tuned for weights
MXFP4	4	Microscaling FP4 (OCP)	Block of 32 shares one scale; E2M1
NVFP4	4	Blackwell training	NVIDIA 4-bit float format

More bits = more fidelity; fewer bits = less memory. Mixed precision & quantisation trade between them.

11

QLoRA, Distillation & What's Next

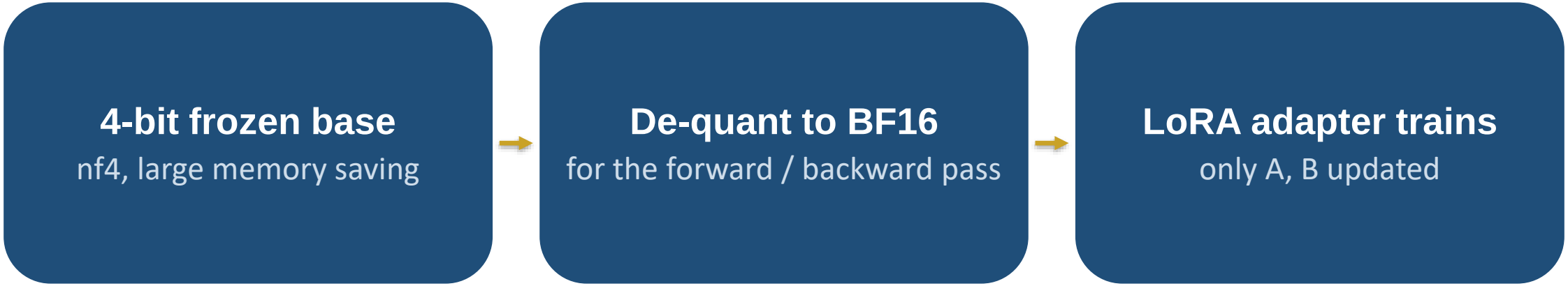
LoRA + quantisation, the recipe behind today's single-GPU fine-tuning, and where it goes.

QLoRA = LoRA + quantisation

- QLoRA builds on LoRA for further memory savings via quantisation
- The frozen base model is held in 4-bit integers → large memory reduction vs. LoRA
- Trade-off: longer training and somewhat degraded quality
- Weights are de-quantised to BF16/FP32 for the forward/backward pass, then re-quantised to 4-bit
- Uses its own 4-bit type, nf4 ("normal float"), tuned for normally-distributed weights
- Newer 4-bit formats push further: MXFP4 (microscaling FP4) let gpt-oss run a 120B model on a single 80GB GPU, a block of 32 values shares one scale

The QLoRA recipe

when frozen store in quantam form



The base stays 4-bit in memory, de-quantised on the fly per layer, the small LoRA adapter carries the learning.

Distillation IS fine-tuning, with generated data

- Not a separate technique: it is ordinary SFT (§4), same loop, same loss
- The ONLY change is where the data comes from: another model writes it
- The "teacher" is a model you RENT, not one you own or pre-train
- Call it a few hundred times → input/output pairs, no human labelling
- Fine-tune a small open student (Llama-8B, Qwen) on those pairs, as in §4
- Result: 5–30× cheaper inference at near-teacher quality on a narrow task
- Caveat: check the teacher's terms of service before doing this commercially

Key takeaways

01 **Fine-tune last, not first**

Climb the ladder: evals → prompting & context → fine-tune. Fine-tune for a behaviour gap, not a knowledge gap.

02 **Beyond SFT**

Post-training now goes SFT → DPO (preferences) → GRPO / RFT (reward-based, often verifiable). TRL ships all three.

03 **PEFT / LoRA is the substrate**

Freeze the base, train a low-rank adapter ($\Delta W = BA$); it underpins SFT, DPO and GRPO alike.

04 **Precision as a lever**

Mixed precision (FP16/BF16) + quantisation (int8, fp8, nf4) → QLoRA fine-tunes billions of params on one GPU.

LAB

Hands-on lab: walk through, then change one thing

- In class: we walk through Topic4-unsloth.ipynb, already run, outputs in place
- It LoRA-tunes a small instruct model (r=16), base loaded in 4-bit
- Your turn: change ONE hyper-parameter, re-run it, and see what moves
  - rank r, learning rate, epochs, batch size / gradient accumulation steps
  - SFT args like max_seq_length live in SFTConfig, not TrainingArguments
- Record: trainable params (%), peak GPU memory, step time, final loss
- Keep the before/after generations on the fixed prompts
- Bring it to the start of the next class, and we compare across the room

Reality check: is fine-tuning even worth it?

- Now that you have run one: most teams never fine-tune, and that is fine
- A good prompt against a strong API is the right default for most tasks
- The barrier is low: a few hundred clean examples change style, format or tone
- Classification is the exception that scales: budget per class, not per dataset
- Cost is a ~$10 run on one rented GPU; the gate is a behaviour gap, not budget
- It has narrowed, not died: a minority path for cost, latency or data residency